

Навигация

Гористая часть Тайваня насчитывает более 250 вершин выше 3000 метров. Эй-Мин планирует создать сети роботизированных зиплайнов для доставки в Центральном горном хребте Тайваня, чтобы перевозить грузы между отдаленными деревнями.

В плане Эй-Мина каждая сеть состоит из N узлов, пронумерованных от 0 до $N - 1$. Среди этих узлов **ровно** $K = 6$ являются станциями зарядки, а остальные $N - K$ узлов являются деревнями. Узлы соединены $N - 1$ **двунаправленными** кабелями, пронумерованными от 0 до $N - 2$. Для каждого i ($0 \leq i \leq N - 2$) кабель i соединяет узлы $U[i]$ и $V[i]$. Каждый кабель соединяет два разных узла, и каждая пара узлов соединена не более чем одним кабелем. Гарантируется, что между любой парой узлов можно перемещаться по кабелям.

Расстояние между узлами a и b обозначается $d(a, b)$ и определяется следующим образом.

- Если $a = b$, то $d(a, b) = 0$.
- В противном случае $d(a, b)$ — это минимальное количество кабелей, необходимое для перемещения от a к b .

Мы говорим, что **коэффициент ветвления** сети не менее δ , если каждый узел сети соединен ровно с 1 или не менее чем с δ кабелями. У Эй-Мина есть информация, что коэффициент ветвления сети не менее целого положительного числа B .

Эй-Мин подготовил 100 различных типов кабелей, пронумерованных $0, 1, \dots, 99$. Каждый кабель в сети будет принадлежать к одному из этих типов.

Роботы будут ездить по кабелям зиплайнов, чтобы выполнять заказы по доставке. Эй-Мин хотел бы установить навигационную программу, которая помогает роботам возвращаться на станции для подзарядки. Однако у роботов очень мало памяти. Поэтому при разработке навигационной программы роботы ориентируются только на количество электростанций $K = 6$, гарантированный коэффициент ветвления B и типы кабелей, подключенных к текущему местоположению робота.

Когда робот хочет переместиться из узла-деревни u , соединенного с двумя или более кабелями, робот сначала сканирует кабели, подключенные к u , в произвольном порядке. Затем программе предоставляется список, содержащий типы кабелей в этом порядке. Для каждого из кабелей программе нужно подсчитать количество станций зарядки, для которых

при движении робота по данному кабелю расстояние между роботом и станцией уменьшается.

В частности, пусть $v_1, \dots, v_k$ ($k \geq 2$) — узлы, непосредственно соединенные с u кабелем, и c_i ($1 \leq i \leq k$) — тип кабеля, соединяющего узел u и v_i . Программе предоставляются K , B и список $c_1, c_2, \dots, c_k$. Для каждого i ($1 \leq i \leq k$), программа должна определить количество станций зарядки p , таких что $d(v_i, p) < d(u, p)$.

Ваша задача — разработать стратегию назначения типов кабелей и создания навигационной программы. Оценка вашего решения зависит от количества использованных различных типов кабелей (подробности см. в разделе «Подзадачи и оценка»).

Детали реализации

Вам необходимо реализовать две функции: одну для назначения типов кабелей и другую для навигационной программы роботов.

Функция, которую вы должны реализовать для **назначения типов кабелей**:

```
std::vector<int> construct_network(int N, int K, int B,  
                                  std::vector<int> U, std::vector<int> V,  
                                  std::vector<int> P)
```

- N : количество узлов.
- K : количество станций зарядки.
- B : гарантированный минимальный коэффициент ветвления сети.
- U, V : массивы длины $N - 1$, задающие соединения кабелей.
- P : массив длины $K = 6$, задающий узлы-станции зарядки.
- Эта функция вызывается **не более 10 000 раз** для каждого тестового примера.

Функция должна возвращать массив T длины $N - 1$:

- Кабель типа $T[i]$ используется для соединения узлов $U[i]$ и $V[i]$.
- Каждый элемент $T[i]$ должен удовлетворять условию $0 \leq T[i] < 100$.

Функция, которую вы должны реализовать для **навигационной программы роботов**:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : количество станций зарядки.
- B : гарантированный минимальный коэффициент ветвления сети.

- C : список типов всех кабелей, подключенных к **некоторому узлу-деревне, из которого идут как минимум 2 кабеля**, в произвольном порядке.
- Гарантируется, что длина C не менее B .
- Эта функция вызывается не более 100 000 раз для каждого тестового примера.
- Функция `navigate` не может полагаться на исходную структуру сети; в частности, проверяющая система может переупорядочить все вызовы `navigate` для разных построенных сетей в рамках одного тестового примера.

Функция должна возвращать массив D :

- Длина массива D должна совпадать с длиной массива C .
- $D[i]$ должно быть количеством станций зарядки таких, что при движении по кабелю, соответствующему $C[i]$, расстояние между роботом и станцией уменьшается.

Обратите внимание, что в проверяющей системе функции `construct_network` и `navigate` вызываются **в двух отдельных процессах**.

Ограничения

- $K = 6$.
- $K < N \leq 100\,000$.
- Сумма N по всем вызовам `construct_network` не превышает 100 000 в каждом тестовом примере.
- $0 \leq U[i] < V[i] < N$ для каждого $0 \leq i \leq N - 2$.
- Можно перемещаться от любого узла к любому другому узлу по кабелям.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- Каждый массив C является перестановкой списка типов всех кабелей, ведущих из какого-то узла-деревни, из которого идёт как минимум 2 кабеля.
- Сумма длин массива C по всем вызовам `navigate` не превышает 200 000 в каждом тестовом примере.

Подзадачи и оценка

Подзадача	Оценка	Дополнительные ограничения
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ для каждого i такого, что $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

Если в любом из тестовых примеров выполняется хотя бы одно из следующих условий, то оценка вашего решения для этого тестового примера будет 0 (в CMS будет сообщено «Output isn't correct»):

- Возвращаемое значение любого вызова `construct_network` недействительно.
- Возвращаемое значение любого вызова `navigate` неверно.

В противном случае пусть S — наименьшее целое число, превышающее все числа в каждом массиве T , возвращенном из каждого вызова `construct_network`. Другими словами, S — это наименьшее целое число, такое что все построенные сети используют только кабели типов $0, 1, \dots, S - 1$.

Ваша оценка за каждую подзадачу зависит от S следующим образом:

Условие	Подзадача 1	Подзадача 2	Подзадачи 3 и 4	Подзадача 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

В частности, в подзадачах 1, 2 и 5 вы получите полный балл, если $S \leq 5$, а в подзадачах 3 и 4 — полный балл, если $S \leq 4$.

Пример

Рассмотрим сценарий, в котором $N = 9$, $K = 6$ и $B = 2$.

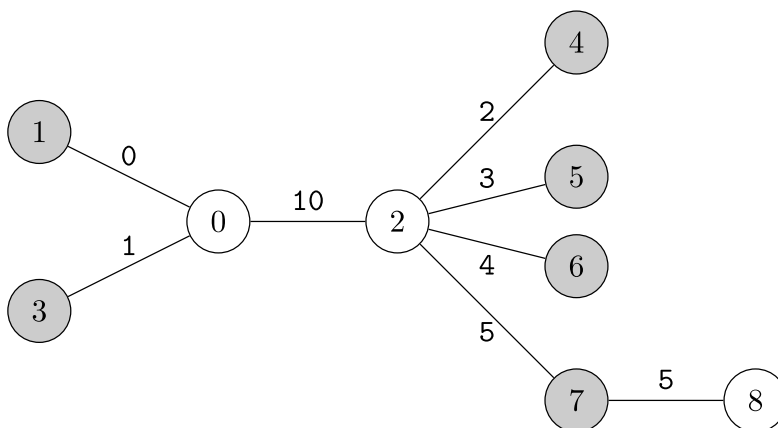
План сети задается $U = [0, 0, 0, 2, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$. Станции подзарядки: $P = [1, 3, 4, 5, 6, 7]$. Рассмотрим следующий вызов первого процесса:

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 2, 7],  
                  [1, 2, 3, 4, 5, 6, 7, 8],  
                  [1, 3, 4, 5, 6, 7])
```

Предположим, Эй-Мин строит сеть, возвращая:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

Построенная сеть проиллюстрирована ниже.



Затем во втором процессе предположим, что роботу нужно переместиться из узла 0. Узел 0 соединен с узлами 1, 2 и 3. Если робот считывает типы кабелей в таком порядке, будет выполнен следующий вызов:

```
navigate(6, 2, [0, 10, 1])
```

Основываясь на структуре сети:

- Первая станция зарядки, узел 1, находится на кратчайшем расстоянии от себя. В частности, $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$.
- Вторая станция зарядки, узел 3, находится на кратчайшем расстоянии от себя. В частности, $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$.
- Другие станции зарядки, узлы 4, 5, 6 и 7, находятся на меньшем расстоянии от узла 2. В частности, $1 = d(2, u) < 2 = d(0, u) < 3 = d(1, u) = d(3, u)$, когда $u = 4, 5, 6$, или 7.

Количество станций зарядки, расстояние до которых уменьшается после перемещения по кабелям (0,1), (0,2) и (0,3), равно 1, 4 и 1 соответственно. Следовательно, функция должна вернуть:

```
[1, 4, 1]
```

Обратите внимание, что `navigate` также может быть вызвана для различных перестановок C . Например, `navigate(6, 2, [1, 0, 10])` также является возможным вызовом для узла 0. Функция должна вернуть `[1, 1, 4]` соответственно.

Предположим, роботу нужно переместиться из узла 2. Выполняется следующий вызов:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

Функция должна вернуть:

```
[2, 1, 1, 1, 1]
```

В этой сети функция `navigate` **никогда** не будет вызываться для других узлов, поскольку:

- узлы 1,3,4,5,6 и 7 являются станциями зарядки, и
- узел 8 соединен только с одним кабелем.

В этом тестовом примере используются типы кабелей 0,1,2,3,4,5, и 10. Таким образом, $S = 11$ будет использоваться для определения оценки.

Аттачмент к этой задаче содержит два других примера входных и выходных данных для выданного участникам грейдера в дополнение к этому примеру.

Грейдер для участников

Выданный участникам грейдер вызывает как `construct_network`, так и `navigate` в одном процессе. Кроме того, когда грейдер для участников вызывает `navigate`, типы кабелей перечисляются в порядке появления соответствующих кабелей во входных данных. Оба этих поведения отличаются от того, что происходит в системе при проверке.

Формат ввода:

```
T
(сценарий 0)
(сценарий 1)
...
(сценарий T-1)
```

Каждый сценарий имеет следующий формат:

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

В каждом сценарии грейдер для участников вызовет `navigate` Q раз, причем i -й вызов с информацией об узле $X[i - 1]$. Обратите внимание, что грейдер для участников поддерживает установку K в целые числа, отличные от 6.

Формат вывода:

Если какой-либо из возвращаемых массивов `construct_network` или `navigate` некорректен, грейдер для участников прерывается и выводит причину. В противном случае грейдер для участников выводит каждый массив D , возвращенный из вызова `navigate`, в отдельной строке.

После обработки всех сценариев грейдер для участников выводит строку в стандартный поток ошибок:

```
S = S'
```

где S' — наименьшее целое число, превышающее все числа в каждом массиве T , возвращенном из каждого вызова `construct_network` в тестовом примере.